



나의 작은 스마트팜 만들기

Wemos D1 R32(ESP32) × 스마트팜 키트 × 구글 시트 × AI Studio 앱

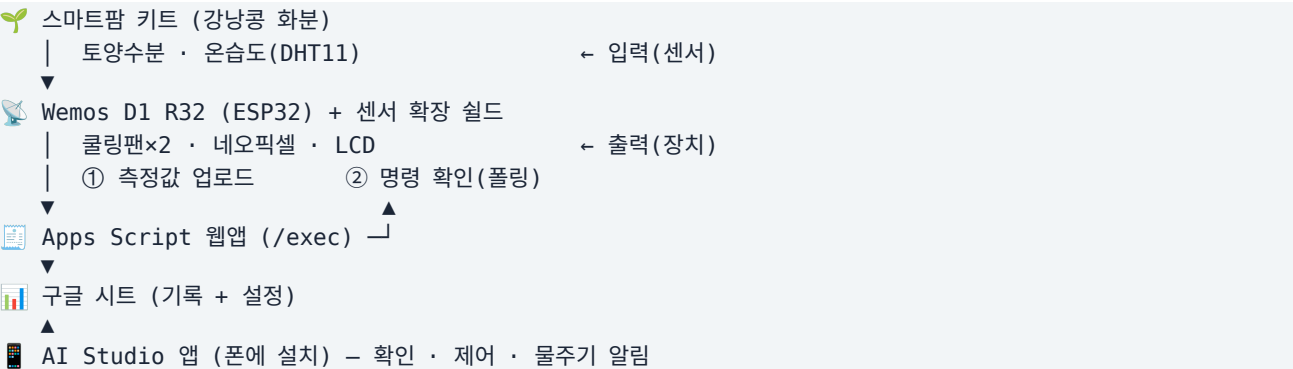
4 차시 워크북 — 읽고, 따라 하고, 기록하며 완성해요

팀명: _____ 이름: _____

들어가며 — 우리가 만드는 것

여러분은 4 차시 동안 '식물을 스스로 돌보는 화분'을 만듭니다. 흙이 마르면 알려주고, 더우면 팬을 돌리고, 추우면
생장등을 켜고, 모든 기록이 구글 시트에 쌓이고, 스마트폰 앱으로 어디서든 확인·조종할 수 있는 진짜 IoT 시스템이에요.

시스템 전체 구조



준비물

- Wemos D1 R32 보드 + USB 케이블 (데이터 전송용!)
- 스마트팜 키트: 센서 확장 쉘드, 2 채널 모터드라이버, 쿨링팬×2, 토양수분 센서, DHT11 온습도 센서, 네오픽셀 LED, LCD, MDF 케이스, 배터리 홀더
- 노트북(Arduino IDE 설치) · 스마트폰 · 구글 계정 · 물컵과 화분

⚠ 보드 설정: 툴 → 보드 → esp32 → ESP32 Dev Module / 시리얼 모니터 속도는 항상 115200! 라이브러리 4 개
설치: DHT sensor library(Adafruit) · Adafruit NeoPixel · LiquidCrystal I2C. "avr 아키텍처" 경고는 무시해도 됩니다.

가장 중요한 표 — 어디에 꽂고, 코드엔 몇 번을 쓸까?

키트 강의자료는 아두이노 우노 기준이라, 우리 보드(D1 R32)에서는 꽂는 줄과 코드의 번호가 달라요. 이 표가 이 워크북의 지도입니다:

부품	꽂는 자리	코드의 번호	주의
토양수분 센서	확장셸드 A2 줄	35	키트 자료의 A1 줄 아님!
온습도(DHT11)	확장셸드 D2 줄	26	키트 자료의 A0 줄 아님!
쿨링팬 (모터드라이버)	D4·D5·D6·D7 줄	17·16·27·14	자리 그대로, 번호만 변환
네오픽셀 LED	D9 줄	13	번호만 변환
LCD (I2C)	보드의 SDA·SCL 핀에 직접	주소 0x27	셸드 A4·A5 줄은 동작 안 함!
블루투스 모듈	사용 안 함	—	WiFi 내장이라 불필요

⚠ 아날로그 값의 범위도 달라요: 우노 0~1023 → ESP32 는 0~4095!

1 차시 — 센서 점검: 흙과 공기를 숫자로 읽기

오늘의 목표: 토양수분 센서와 온습도 센서를 연결해 시리얼 모니터로 값을 읽고, 우리 센서에 맞는 보정값을 찾는다.

실습 1. 토양수분 센서 — 흙의 목마름을 숫자로

배선: G(A2)→GND · V(A2)→VCC · S(A2)→AO (확장шил드 A2 줄 = GPIO35)

 사진 자리: 토양수분 센서를 확장шил드 A2 줄에 연결한 모습 (클로즈업)

△ 센서의 전원부(위쪽 기판)에 물이 닿으면 고장나요! 물에 담글 땐 아래쪽 막대 부분만.

코드 (code/01_soil/01_soil.ino):

```
/*
1 단계 ① 토양수분 감지 모듈 — 흙의 목마름을 숫자로!
△ 키트 자료에는 A1로 나오지만, D1 R32에서는 A1 자리(GPIO4)가 WiFi와 겹쳐요.
케이블을 확장шил드의 **A2 줄**에 꽂으세요 → GPIO35
결선: G(A2)→GND · V(A2)→VCC · S(A2)→AO
주의: 센서의 전원부(위쪽 기판)에 물이 닿지 않게!

★ 이 키트의 센서는 "마를수록 값이 커지는" 타입으로 확인됐어요 (공기 중 ≈ 4095, 물속 ≈ 950).
'2점 보정' 방식이라 마른 값·젖은 값을 직접 재서 넣으면
0%(바싹 마름) ~ 100%(흠뻑 젖음)가 올바르게 나옵니다.
*/

// 핀 번호와 보정값을 이름으로 정해둔다
#define SOIL 35 // 토양수분 센서가 연결된 아날로그 핀 (확장шил드 A2 줄 = GPIO35)
#define SOIL_DRY_RAW 4095 // ★ 공기 중(바싹 마름)에서 시리얼로 켄 '아날로그' 값 (이 키트 센서는 마를수록 커요!)
#define SOIL_WET_RAW 950 // ★ 물컵에 담갔을 때 켄 '아날로그' 값 — 내 센서 값으로 바꾸세요!

// 처음 한 번만 실행
void setup() {
  Serial.begin(115200); // 시리얼 통신 시작 (시리얼 모니터도 115200으로!)
}

// 무한 반복
void loop() {
  int raw = analogRead(SOIL); // 센서의 아날로그 값 읽기: 0~4095 (우노는 0~1023이었어요!)
  // map(마른 값 → 0%, 젖은 값 → 100%): 두 기준점 사이에서 비율 계산
  // 젖은 값이 마른 값보다 작음(반대 방향) 센서여도 map이 알아서 뒤집어 계산해요!
  // constrain(): 보정 범위를 벗어난 값은 0~100 안으로 잘라내기
  int pct = constrain(map(raw, SOIL_DRY_RAW, SOIL_WET_RAW, 0, 100), 0, 100);
  Serial.printf("아날로그 %d → 토양습도 %d%%\n", raw, pct); // 두 값을 함께 출력
  delay(500); // 0.5초마다 측정
}

/*
[2점 보정 순서 — 딱 두 번만 재면 돼요]
1. 이 코드를 그대로 업로드하고 시리얼 모니터(115200)를 켜요
2. 센서를 공기 중에 두고 '아날로그' 값 확인 → SOIL_DRY_RAW 에 입력
3. 센서 아래쪽만 물컵에 담그고 '아날로그' 값 확인 → SOIL_WET_RAW 에 입력
(△ 위쪽 기판까지 담그면 안 돼요!)
4. 재업로드 → 공기 중 0% / 물속 100% 가 나오면 보정 완료!
5. 이 두 값을 05·06 코드의 같은 이름 칸에도 복사하세요.

[진단 — 값이 이상할 때]
- '아날로그' 값이 공기/물에서 확실히 달라진다 → 센서 정상! 두 값만 위에 넣으면 끝
- '아날로그' 값이 아예 안 변한다(계속 0 또는 4095 근처)
→ 센서가 아니라 배선 문제! 신호선(AO)이 확장шил드 A2 줄의 S 칸에 꽂혔는지,
G·V가 제대로 연결됐는지 확인하세요 (핀이 붕 뜨면 4095 근처가 계속 나와요)
*/
```

활동 ① 2점 보정 — 우리 센서의 기준 찾기

이 키트의 센서는 '마를수록 값이 커지는' 타입이에요. 두 상황의 아날로그 값을 재서 기록하고, 코드의 두 칸에 넣으세요:

상황	시리얼에 나온 '아날로그' 값	코드에 넣을 칸
공기 중 (바싹 마름)		SOIL_DRY_RAW
물컵에 담금 (흠뻑 젖음)		SOIL_WET_RAW
물 준 화분의 흙		(관찰만 — 몇 %가 나왔나요? %)

💡 보정 후 공기 중 0%, 물속 100%가 나오면 성공! 이 두 값은 3·4 차시 코드에도 그대로 복사합니다.

실습 2. 온습도 센서(DHT11) — 공기의 상태

배선: G(D2)→GND · V(D2)→VCC · S(D2)→DATA (확장실드 D2 줄 = GPIO26)

📷 사진 자리: DHT11 을 확장실드 D2 줄에 연결한 모습

코드 (code/02_dht/02_dht.ino):

```
/*
1 단계 ② 온·습도 센서(DHT11) — 스마트팜 속 공기 상태
△ 키트 자료에는 A0 로 나오지만, D1 R32 에서 A0 자리(GPIO2)는 부팅 문제를 일으킬 수 있어요.
케이블을 확장실드의 **D2 줄**에 꽂으세요 → GPIO26
결선: G(D2)→GND · V(D2)→VCC · S(D2)→DATA
라이브러리: DHT sensor library (Adafruit)
*/

#include <DHT.h>          // 온습도 센서 제어용 라이브러리 불러오기

#define DHTPIN 26         // DHT11 센서가 연결된 핀 (확장실드 D2 줄 = GPIO26)

DHT dht(DHTPIN, DHT11);  // 온습도 센서 객체 생성 (핀 번호, 센서 종류)

// 처음 한 번만 실행
void setup() {
  Serial.begin(115200);    // 시리얼 통신 시작
  dht.begin();            // 온습도 센서 작동 시작
}

// 무한 반복
void loop() {
  float t = dht.readTemperature();  // 섭씨 온도 읽기 (float = 소수점 있는 숫자)
  float h = dht.readHumidity();     // 습도(%) 읽기

  // isnan = "숫자가 아님(Not a Number)" 검사 — 측정에 실패하면 nan 이 나와요
  if (isnan(t) || isnan(h)) {
    Serial.println("측정 실패 — 배선(G·V·S)과 핀 번호를 확인하세요");
  } else {
    Serial.printf("온도 %.1f °C / 습도 %.0f %%\n", t, h); // 온도는 소수점 1자리, 습도는 정수로 출력
  }
  delay(2000);    // DHT11 은 느린 센서라 2 초에 한 번이 적당해요
}
```

✅ 확인: 센서를 손으로 감싸면 온도가, 입김을 불면 습도가 올라가나요? 지금 교실은 온도 _____ °C / 습도 _____ %

🔧 1 차시 문제해결소

증상	해결
포트(COM)가 없어요	충전 전용 케이블인지 확인 → 데이터 케이블로. CH340 드라이버 설치
업로드 시 포트가 사용 중이라요	시리얼 모니터 창을 모두 닫고 다시 업로드
아날로그 값이 계속 0 또는 4095	신호선이 A2 줄의 S 칸에 제대로 꽂혔는지, G·V 확인 (핀이 봉 뜨면 4095 근처 고정)
온습도가 '측정 실패'	배선(G·V·S) 순서와 핀 번호 26 확인, 2 초에 한 번만 읽기

2 차시 — 출력 점검과 조립: 팬·빛·화면

오늘의 목표: 쿨링팬·네오피셀·LCD 를 하나씩 작동시키고, 키트를 조립해 스마트팜의 몸을 완성한다.

실습 3. 쿨링팬 — 모터드라이버로 바람 만들기

배선: 확장실드 S(4)→A-1B · S(5)→A-1A · S(6)→B-1A · S(7)→B-1B / G(4)→GND · V(4)→VCC / 팬은 모터드라이버의 MOTOR A·B 단자에 (빨간선·검은선 위치 주의!)

 사진 자리: 모터드라이버와 팬 2 개 결선 모습

코드 (code/03_fan/03_fan.ino):

```
/*
  2 단계 ① 쿨링 팬 — 2 채널 모터 드라이버로 팬 2 개 돌리기
  결선(키트 조립 그대로): 확장실드 S(4)→A-1B · S(5)→A-1A · S(6)→B-1A · S(7)→B-1B
                        G(4)→GND · V(4)→VCC / 팬은 모터 드라이버 MOTOR A·B 단자에
  D1 R32 변환: D4→17, D5→16, D6→27, D7→14 ← 코드의 번호는 이것!
  △ 팬이 안 돌면 배터리(외부 전원)를 연결하세요 — USB 전원만으론 부족할 수 있어요
*/

// 모터 드라이버 제어 핀 4 개 — 팬 하나에 핀 2 개씩 (방향 결정용)
#define AA 16 // 팬 A 방향 1 (확장실드 D5 줄 = GPIO16)
#define AB 17 // 팬 A 방향 2 (확장실드 D4 줄 = GPIO17)
#define BA 27 // 팬 B 방향 1 (확장실드 D6 줄 = GPIO27)
#define BB 14 // 팬 B 방향 2 (확장실드 D7 줄 = GPIO14)

// 처음 한 번만 실행
void setup() {
  // 네 핀 모두 "출력용"으로 설정 — 보드가 이 핀으로 전기 신호를 내보낼 거예요
  pinMode(AA, OUTPUT);
  pinMode(AB, OUTPUT);
  pinMode(BA, OUTPUT);
  pinMode(BB, OUTPUT);
}

// 무한 반복
void loop() {
  // 팬 두 개 켜기 — 한쪽 핀은 HIGH(전기 0), 반대쪽 핀은 LOW(전기 X)여야 회전!
  // (두 핀이 같으면 멈춰요. HIGH/LOW를 서로 바꾸면 반대 방향으로 돌아요)
  digitalWrite(AA, HIGH); digitalWrite(AB, LOW); // 팬 A 회전
  digitalWrite(BA, HIGH); digitalWrite(BB, LOW); // 팬 B 회전
  delay(5000); // 5 초 동안 돌리기

  // 팬 두 개 끄기 — 모든 핀 LOW
  digitalWrite(AA, LOW); digitalWrite(AB, LOW); // 팬 A 정지
  digitalWrite(BA, LOW); digitalWrite(BB, LOW); // 팬 B 정지
  delay(5000); // 5 초 동안 정지
}

/*
  [응용 — 키트 자료의 속도 3 단계 실습]
  digitalWrite 대신 analogWrite를 쓰면 속도 조절(PWM)이 돼요:
    analogWrite(AA, 100); → 느리게
    analogWrite(AA, 150); → 중간
    analogWrite(AA, 255); → 최대
  (AB는 LOW 유지. 값 범위 0~255 — 값이 클수록 빨라요)
*/
```

△ 팬이 힘없이 돌거나 안 돌면 배터리 홀더로 외부 전원을 연결하세요. USB 전원만으론 부족할 수 있어요.

✅ 도전: analogWrite 로 속도 3 단계(100/150/255)를 만들어 보세요. 어느 값부터 팬이 돌기 시작하나요? _____

실습 4. 네오피셀 — 식물 생장등

배선: G(9)→GND · V(9)→VCC · S(9)→IN (확장실드 D9 줄 = GPIO13)

코드 (code/04_neopixel/04_neopixel.ino):

```
/*
  2 단계 ② 네오피셀 RGB LED — 식물 생장등 & 상태등
  결선(키트 그대로): 확장실드 G(9)→GND · V(9)→VCC · S(9)→IN
  D1 R32 변환: D9 → GPIO13 ← 코드의 번호는 이것!
  라이브러리: Adafruit NeoPixel
*/
```

```

△ 밝기를 너무 높이면 모듈이 뜨거워져요 — setBrightness 200 이하 권장
*/

#include <Adafruit_NeoPixel.h>    // 네오피셀 LED 제어용 라이브러리 불러오기

#define PIN    13    // 네오피셀이 연결된 핀 (확장실드 D9 줄 = GPIO13)
#define NUMLED 12    // 모듈에 달린 LED 알갱이 개수 (우리 모듈은 12 개)

// 네오피셀 객체 생성 (LED 개수, 핀 번호, 색 순서와 통신 속도 설정)
Adafruit_NeoPixel led(NUMLED, PIN, NEO_GRB + NEO_KHZ800);

// 모든 LED를 한 가지 색으로 바꾸는 함수 (r, g, b = 빨강·초록·파랑의 세기, 각 0~255)
void setColor(int r, int g, int b) {
    for (int i = 0; i < NUMLED; i++)          // 12 개의 LED를 하나씩 돌면서
        led.setPixelColor(i, led.Color(r, g, b));    // i 번째 LED에 색 지정 (아직 화면엔 반영 안 됨)
    led.show();                                // 지정한 색을 실제 LED에 반영!
}

// 처음 한 번만 실행
void setup() {
    led.begin();                                // 네오피셀 작동 시작
    led.setBrightness(150);    // 전체 밝기 설정 (0~255 - 너무 높으면 뜨거워요)
}

// 무한 반복 — 색이 1초씩 바뀌어요
void loop() {
    setColor(255, 0, 0);    delay(1000);    // 빨강 (R만 최대)
    setColor(0, 255, 0);    delay(1000);    // 초록 (G만 최대)
    setColor(0, 0, 255);    delay(1000);    // 파랑 (B만 최대)
    setColor(255, 150, 60);    delay(2000);    // 따뜻한 빛 (빨강+초록 조금+파랑 조금 = 주황빛)
    setColor(255, 0, 255);    delay(2000);    // 보라색 (빨강+파랑 = 실제 식물 성장등 색!)
    setColor(0, 0, 0);    delay(1000);    // 끄기 (모든 색 0)
}

```

💡 식물 성장등이 보라색인 이유: 식물은 빨간 빛과 파란 빛을 주로 사용하고 초록 빛은 반사해요(그래서 잎이 초록색!).
 빨강+파랑 = 보라색.

실습 5. LCD — 스마트팜의 작은 상황판

△ 확장실드의 A4·A5 줄에 꽂으면 안 돼요! LCD의 SDA·SCL 선은 보드 위쪽의 SDA(21)·SCL(22) 핀에 직접 연결합니다. GND·VCC는 실드의 G·V 아무 줄이나 OK.

📷 사진 자리: LCD의 SDA·SCL을 보드 핀에 직접 연결한 모습 (클로즈업)

코드 (code/08_lcd/08_lcd.ino):

```

/*
2 단계 ③ LCD (I2C) — 스마트팜의 작은 상황판
△ 확장실드의 A4·A5 줄에 꽂으면 안 돼요! (D1 R32에서는 그 자리가 입력 전용)
LCD의 SDA·SCL 선을 **보드 위쪽의 SDA·SCL 핀에 직접** 연결하세요.

결선: LCD GND → 실드 G (아무 줄) / LCD VCC → 실드 V (아무 줄)
LCD SDA → 보드 SDA 핀 (GPIO21) ← 직접!
LCD SCL → 보드 SCL 핀 (GPIO22) ← 직접!
라이브러리: LiquidCrystal I2C ("avr 아키텍처" 경고는 무시해도 OK)
*/

#include <LiquidCrystal_I2C.h>    // I2C 방식 LCD 제어용 라이브러리 불러오기

// LCD 객체 생성 (I2C 주소, 가로 칸수, 세로 줄수)
LiquidCrystal_I2C lcd(0x27, 16, 2);    // 키트 LCD 주소 = 0x27 (안 나오면 0x3F로)

int count = 0;    // 숫자를 담아둘 변수 (상자) — 1초마다 1씩 커져요

// 처음 한 번만 실행
void setup() {
    lcd.init();                                // LCD 초기화 (통신 시작)
    lcd.backlight();                            // 백라이트(화면 불빛) 켜기
    lcd.setCursor(0, 0);                        // 커서를 첫째 줄(0), 첫째 칸(0)으로
    lcd.print("Smart Farm!");    // 글자 출력 (영문·숫자만 — 한글은 안 나와요!)
}

// 무한 반복
void loop() {
    lcd.setCursor(0, 1);                        // 커서를 둘째 줄(1) 처음으로
    lcd.print("count: ");                        // 글자 출력
    lcd.print(count);                            // 변수에 담긴 숫자 출력
    count = count + 1;                            // 숫자를 1 키우고
}

```

```
    delay(1000);                // 1초 쉬고 다시 (숫자가 실시간으로 변해요 = '갱신')
}

/*
[안 나올 때]
1. 화면에 불은 켜지는데 글자가 없다 → 뒷면 파란 다이얼(대비)을 드라이버로 살짝 조절
2. 주소 문제 → 0x27을 0x3F로 바꿔 재업로드
3. 그래도 안 되면 → esp32-class-projects/08_cell_samples/00_i2c_scanner 로 실제 주소 확인
*/
```

키트 조립

MDF 케이스 조립과 결선은 키트 강의자료 PDF의 '스마트 팜 키트 조립하기'를 그대로 따라 하세요. 달라지는 곳은 딱 두 곳 — 토양수분은 A2 줄, DHT는 D2 줄!

 사진 자리: 조립 완성된 우리 팀 스마트팜 전체 모습

3 차시 — 자동 스마트팜: 스스로 판단하는 화분

오늘의 목표: 센서 값에 따라 팬과 생장등이 스스로 켜지고 꺼지는 자동 제어를 완성한다. (인터넷 없이!)

자동 제어 규칙 — 강남콩 발아 기준

조건	동작	왜?
온도 > 26°C	팬 ON	발아 적정온도(21.6~25.8°C) 초과
온도 < 21°C	생장등(따뜻한 빛) ON	한파 피해 방지
습도 > 80%	팬 ON	곰팡이 방지 환기
토양습도 > 70%	생장등 ON	과습 말리기
토양습도 < 30%	LCD·시리얼에 "물 주세요!"	펌프가 없으니 사람이 급수!

코드 (code/05_auto_smartfarm/05_auto_smartfarm.ino) — 1 차시에서 찾은

SOIL_DRY_RAW·SOIL_WET_RAW 를 꼭 넣으세요:

```
/*
3 단계 — 자동 스마트팜 (키트 예제 12_SmartFarm_KIT 의 D1 R32 버전, 인터넷 없이!)
강남콩 발아 기준으로 온도·습도·토양습도를 읽어 팬과 LED 를 스스로 제어해요.

— 자동 규칙 (기준값은 우리 화분에 맞게 조절!) —
온도 26°C 초과 → 팬 ON (더위 식히기)
온도 21°C 미만 → 네오피셀 따뜻한 빛 (보온)
습도 80% 초과 → 팬 ON (곰팡이 방지 환기)
토양습도 70% 초과 → 네오피셀 따뜻한 빛 (과습 말리기 — 키트 자료 방식)
토양습도 30% 미만 → 시리얼·LCD 에 "물 주세요!" (키트엔 펌프가 없어서 사람이 급수!)

— 배선 (D1 R32 변환 완료) —
토양수분 → A2 자리(GPI035) / DHT11 → D2 자리(GPI026)
팬 → D4·D5·D6·D7 자리(17·16·27·14, 모터드라이버 경유) / 네오피셀 → D9 자리(GPI013)
LCD → SDA·SCL 을 보드 핀(21·22)에 직접! (шил드 A4·A5 줄은 안 돼요)
*/

// 제어에 필요한 라이브러리들을 불러온다
#include <DHT.h> // 온습도 센서 제어용
#include <Adafruit_NeoPixel.h> // 네오피셀 LED 제어용
#include <LiquidCrystal_I2C.h> // I2C 방식 LCD 제어용

// 보드에 연결된 핀 번호들
#define SOIL 35 // 토양수분 센서 (확장실드 A2 줄)
#define DHTPIN 26 // 온습도 센서 (확장실드 D2 줄)
#define AA 16 // 모터드라이버 팬 A 방향 1 (D5 줄)
#define AB 17 // 모터드라이버 팬 A 방향 2 (D4 줄)
#define BA 27 // 모터드라이버 팬 B 방향 1 (D6 줄)
#define BB 14 // 모터드라이버 팬 B 방향 2 (D7 줄)
#define LEDPIN 13 // 네오피셀 (D9 줄)
#define NUMLED 12 // 네오피셀 LED 알갱이 개수

#define SOIL_DRY_RAW 4095 // ★ 01_soil 로 잰 '공기 중(마름)' 아날로그 값 (이 키트 센서는 마름수록 커요!)
#define SOIL_WET_RAW 950 // ★ 01_soil 로 잰 '물속(젖음)' 아날로그 값

// — 자동 제어 기준값 (강남콩 발아 생육정보: 21.6~25.8°C · 습도 50%) —
#define TEMP_HIGH 26 // 이보다 더우면 팬 켜기
#define TEMP_LOW 21 // 이보다 추우면 보온등 켜기
#define HUMI_HIGH 80 // 이보다 습하면 환기(팬)
#define SOIL_WET 70 // 이보다 젖었으면 말리기(보온등)
#define SOIL_DRY 30 // 이보다 마르면 급수 알림

// 센서·모듈 객체 생성
DHT dht(DHTPIN, DHT11); // 온습도 센서 (핀, 종류)
Adafruit_NeoPixel led(NUMLED, LEDPIN, NEO_GRB + NEO_KHZ800); // 네오피셀
LiquidCrystal_I2C lcd(0x27, 16, 2); // LCD (I2C 주소 0x27, 16 칸×2 줄 — 안 나오면 0x3F)

// 팬 두 개를 켜고(true) 끄는(false) 함수
void fan(bool on) {
  // "on ? HIGH : LOW" = 조건 연산자: on이 참이면 HIGH, 거짓이면 LOW
  // 한쪽 핀만 HIGH가 되어야 회전, 둘 다 LOW면 정지
  digitalWrite(AA, on ? HIGH : LOW); digitalWrite(AB, LOW); // 팬 A
```



```

digitalWrite(BA, on ? HIGH : LOW); digitalWrite(BB, LOW);    // 팬 B
}

// 네오픽셀을 따뜻한 빛으로 켜고(true) 끄는(false) 함수
void warmLight(bool on) {
  for (int i = 0; i < NUMLED; i++)                          // LED 12 개를 하나씩
    led.setPixelColor(i, on ? led.Color(255, 150, 60) : 0); // 켜면 따뜻한 주황빛, 끄면 0(꺼짐)
  led.show();                                                // 실제 LED 에 반영
}

// 처음 한 번만 실행되는 초기화 구간
void setup() {
  Serial.begin(115200);                                       // 시리얼 통신 시작
  pinMode(AA, OUTPUT); pinMode(AB, OUTPUT); // 모터드라이버 핀 4 개를 출력용으로
  pinMode(BA, OUTPUT); pinMode(BB, OUTPUT);
  dht.begin();                                                // 온습도 센서 시작
  led.begin();                                                // 네오픽셀 시작
  led.setBrightness(150);                                     // 밝기 150 (최대 255)
  lcd.init();                                                 // LCD 초기화
  lcd.backlight();                                           // LCD 백라이트 켜기
}

// 무한 반복 구간
void loop() {
  // — ① 세 가지 값 측정 —————
  float t = dht.readTemperature(); // 온도(°C)
  float h = dht.readHumidity();    // 습도(%)
  // 2점 보정: 마른 값→0%, 젖은 값→100% (방향이 반대인 센서도 map이 알아서 처리)
  int soilPct = constrain(map(analogRead(SOIL), SOIL_DRY_RAW, SOIL_WET_RAW, 0, 100), 0, 100);
  Serial.printf("T %.1f°C H %.0f%% Soil %d%% → ", t, h, soilPct); // 시리얼에 출력

  // — ② 자동 판단 — 조건을 변수에 담아두기 —————
  // || 는 "또는" - 둘 중 하나만 참이어도 참
  bool needFan = (t > TEMP_HIGH) || (h > HUMI_HIGH); // 덥거나 습하면 팬 필요
  bool needWarm = (t < TEMP_LOW) || (soilPct > SOIL_WET); // 춥거나 과습이면 보온등 필요

  // — ③ 판단 결과대로 장치 작동 —————
  fan(needFan); // 팬 켜기/끄기
  warmLight(needWarm); // 보온등 켜기/끄기

  // — ④ LCD 상황판 — 1줄: 온습도 / 2 줄: 토양습도 + 상태 ———
  lcd.clear(); // 이전 글자 지우기
  lcd.setCursor(0, 0); // 커서를 첫째 줄 처음으로
  lcd.print("T:"); lcd.print(t, 1); // 온도 (소수점 1자리)
  lcd.print(" H:"); lcd.print(h, 0); lcd.print("%"); // 습도 (정수)
  lcd.setCursor(0, 1); // 커서를 둘째 줄 처음으로
  lcd.print("Soil:"); lcd.print(soilPct); lcd.print("% "); // 토양습도
  // 지금 상태를 한 단어로 표시 (위에서부터 차례로 검사 — 먼저 걸리는 것 하나만)
  if (soilPct < SOIL_DRY) lcd.print("WATER!"); // 말랐어요 — 물 주세요!
  else if (needFan) lcd.print("FAN"); // 팬 작동 중
  else if (needWarm) lcd.print("WARM"); // 보온등 작동 중
  else lcd.print("OK"); // 모든 조건 정상

  // — ⑤ 시리얼에도 상태 출력 —————
  if (needFan) Serial.print("팬 ON ");
  if (needWarm) Serial.print("보온등 ON ");
  if (soilPct < SOIL_DRY) Serial.print("💧 물 주세요! ");
  Serial.println(); // 줄바꿈

  delay(2000); // 2 초 쉬고 처음부터 다시
}

```

활동 ② 자동 제어 실험

실험	예상	실제 결과
DHT 를 손으로 감싸 온도 올리기	팬이 돈다	
토양수분 센서를 물컵에 담그기(70% 초과)	생장등이 켜진다	
센서를 공기 중에 빼두기(30% 미만)	LCD 에 WATER!	

💡 기준값(26°C·21°C·80%·70%·30%)은 코드 위쪽 #define 에 모여 있어요. 우리 교실·우리 화분에 맞게 바꿔 보세요 — 이게 진짜 '스마트팜 튜닝'입니다!

4 차시 — 클라우드와 앱: 어디서든 우리 화분

오늘의 목표: 측정값을 구글 시트에 자동 기록하고, AI Studio 로 만든 앱에서 확인·제어한다.

① 구글 시트 + Apps Script 만들기

1. sheets.new → 이름 '스마트팜' / 시트 1 탭 1 행: 시각·토양습도·온도·습도
2. '설정' 탭 추가 → A1 에 AUTO, B1 에 30 입력
3. 확장 프로그램 → Apps Script → 아래 서버 코드 전체 붙여넣기 → 저장
4. 배포 → 새 배포 → 웹 앱 → 액세스 '모든 사용자' → 배포 → /exec 주소 복사

△ 나중에 서버 코드를 고치면? 저장만으론 반영 안 돼요! 배포 → 배포 관리 → 연필(✎) → '새 버전' → 배포. (새 배포를 만들면 주소가 바뀌어 버려요!)

서버 코드 (code/07_apps_script_smartfarm.gs):

```
/**
 * 스마트팜 — Apps Script 웹앱 (전체 코드)
 * 이 코드는 구글 시트에 붙여 "웹 창구(/exec 주소)"를 만들어요.
 * ESP32와 앱은 이 주소로 요청을 보내고, 아래 doGet 이 요청을 받아 처리합니다.
 *
 * 시트 구성: '시트1' 탭(기록) 1행 = 시각·토양습도·온도·습도
 *           '설정' 탭 A1 = 명령(AUTO/FAN_ON/FAN_OFF/LED_ON/LED_OFF)
 *           B1 = 급수 알림 기준 토양습도(%) — 예: 30
 *
 * 주소 사용법 (URL = 배포한 /exec 주소):
 * URL?soil=45&temp=25.3&humi=60      → 기록 저장 + "명령,기준값" 응답 (ESP32 가 30 초마다)
 * URL?mode=cmd                      → "명령,기준값" 읽기 — 예: AUTO,30 (ESP32 가 3 초마다)
 * URL?mode=set&cmd=FAN_ON           → 설정!A1에 명령 쓰기 (앱 버튼)
 * URL?mode=latest                   → 최신 측정값 JSON (앱 화면)
 * URL?mode=limit&value=30           → 설정!B1 기준값 변경 (앱 슬라이더)
 * URL?mode=getlimit                 → 설정!B1 기준값 읽기
 */

// 누군가 웹 주소로 GET 요청을 보낼 때마다 자동으로 실행되는 함수
function doGet(e) {
  const ss = SpreadsheetApp.getActiveSpreadsheet(); // 이 스크립트가 붙어있는 스프레드시트
  const log = ss.getSheetByName("시트1"); // 기록 탭
  const cfg = ss.getSheetByName("설정"); // 설정 탭 (A1=명령, B1=기준값)
  const p = e.parameter; // 주소 뒤 ?이름=값 들이 여기 담겨요 (예: p.soil)

  // — 명령 읽기 (ESP32 폴링) — "명령,기준값" 형태로 한 번에! —
  if (p.mode == "cmd") {
    // A1의 명령과 B1의 기준값을 쉼표로 이어 붙여 응답 (예: "AUTO,30")
    const out = String(cfg.getRange("A1").getValue()) + "," + String(cfg.getRange("B1").getValue());
    return ContentService.createTextOutput(out);
  }

  // — 명령 쓰기 (앱 버튼) —
  if (p.mode == "set") {
    cfg.getRange("A1").setValue(p.cmd); // 설정!A1에 명령 저장 (예: FAN_ON)
    return ContentService.createTextOutput("OK: " + p.cmd);
  }

  // — 급수 알림 기준값 변경/읽기 —
  if (p.mode == "limit") {
    cfg.getRange("B1").setValue(Number(p.value)); // 설정!B1에 숫자로 저장
    return ContentService.createTextOutput("OK: limit=" + p.value);
  }
  if (p.mode == "getlimit") {
    return ContentService.createTextOutput(String(cfg.getRange("B1").getValue()));
  }

  // — 최신 측정값 JSON (앱 화면 표시용) —
  if (p.mode == "latest") {
    const r = log.getLastRow(); // 기록의 마지막 줄 번호
    if (r < 2) return ContentService.createTextOutput("{}"); // 기록이 아직 없으면 빈 응답
    const v = log.getRange(r, 1, 1, 4).getValues()[0]; // 마지막 줄의 4칸(시각·습도) 읽기
    const out = { // 앱이 읽기 좋은 JSON 모양으로 포장
      time: Utilities.formatDate(new Date(v[0]), "Asia/Seoul", "MM-dd HH:mm:ss"),
      soil: v[1], temp: v[2], humi: v[3],
      limit: Number(cfg.getRange("B1").getValue()), // 현재 기준값도 함께
      cmd: String(cfg.getRange("A1").getValue()) // 현재 명령도 함께
    };
    return ContentService.createTextOutput(JSON.stringify(out))
      .setMimeType(ContentService.MimeType.JSON);
  }
}
```

```
// — 기본: 측정값 기록 (ESP32 업로드) —————
// 응답에 "명령,기준값"을 실어 줘요 — 보드가 통신 1 번으로 기록+명령확인을 한 번에!
log.appendRow([new Date(), p.soil, p.temp, p.humi]); // 맨 아래에 새 줄 추가
const reply = String(cfg.getRange("A1").getValue()) + "," + String(cfg.getRange("B1").getValue());
return ContentService.createTextOutput(reply); // 예: "AUTO,30"
}
```

② 브라우저로 먼저 테스트 (보드 없이!)

주소 뒤에 붙일 것	기대 결과
?soil=45&temp=25.3&humi=60	시트 1 에 새 줄 + 응답 AUTO,30
?mode=cmd	AUTO,30
?mode=set&cmd=FAN_ON	설정!A1 이 FAN_ON 으로
?mode=latest	마지막 줄 JSON
?mode=limit&value=35	설정!B1 이 35 로

③ 보드 코드 업로드

코드 (code/06_cloud_smartfarm/06_cloud_smartfarm.ino) — ★ 3 줄(WiFi 이름·비밀번호·URL) 과 보정값 2 개를 내 것으로:

```
/*
4 단계 — 클라우드 스마트팜 (완성판)
일은 세 가지 리듬으로 나눠서 해요 (delay 대신 millis 시계 사용):
2 초마다 → 센서 측정 + LCD + 자동 제어
3 초마다 → 앱 명령 확인 (버튼 반응이 빨라져요!)
30 초마다 → 구글 시트에 기록 (기록은 천천히 쌓여도 충분)

※ 앱 버튼 → 실제 동작까지 3~6 초 정도 걸리는 게 정상이에요.
(앱→시트는 즉시, 보드가 시트를 3 초마다 확인 + 통신 왕복 시간)

— 배선 (05_auto_smartfarm과 동일) —
토양수분 → A2 자리(GPI035) / DHT11 → D2 자리(GPI026)
팬 → 17·16·27·14 (모터드라이버) / 네오픽셀 → D9 자리(GPI013)
LCD → SDA·SCL 을 보드 핀(21·22)에 직접! (쉴드 A4·A5 줄은 안 돼요)

— 명령 (시트 설정!A1 ← 앱이 씬) —
AUTO / FAN_ON / FAN_OFF / LED_ON / LED_OFF

★ 바꿀 곳 3 줄: WIFI_SSID · WIFI_PASS · URL
*/

// 통신 및 센서 제어를 위해 필요한 라이브러리들을 불러온다
#include <WiFi.h> // 와이파이 연결용
#include <HttpClient.h> // 웹(구글 시트) 통신용
#include <DHT.h> // 온습도 센서 제어용
#include <Adafruit_NeoPixel.h> // 네오픽셀 LED 제어용
#include <LiquidCrystal_I2C.h> // I2C 방식 LCD 제어용

// 보드에 연결된 핀 번호들
#define SOIL 35 // 토양수분 센서 (확장실드 A2 줄)
#define DHTPIN 26 // 온습도 센서 (확장실드 D2 줄)
#define AA 16 // 모터드라이버 팬 A 방향 1 (D5 줄)
#define AB 17 // 모터드라이버 팬 A 방향 2 (D4 줄)
#define BA 27 // 모터드라이버 팬 B 방향 1 (D6 줄)
#define BB 14 // 모터드라이버 팬 B 방향 2 (D7 줄)
#define LEDPIN 13 // 네오픽셀 (D9 줄)
#define NUMLED 12 // 네오픽셀 LED 알갱이 개수

// 자동 제어 기준값
#define SOIL_DRY_RAW 4095 // ★ 01_soil 로 켜 '공기 중(마름)' 아날로그 값 (이 키트 센서는 마를수록 커요!)
#define SOIL_WET_RAW 950 // ★ 01_soil 로 켜 '물속(젖음)' 아날로그 값
#define TEMP_HIGH 26 // 이보다 더우면 팬
#define TEMP_LOW 21 // 이보다 추우면 보온등
#define HUMI_HIGH 80 // 이보다 습하면 환기(팬)
#define SOIL_WET 70 // 이보다 젖었으면 따뜻한 빛으로 말리기

// 와이파이·구글 시트 연결 정보 — ★ 내 것으로 바꾸세요!
```

```

const char* WIFI_SSID = "WiFi 이름"; // ★ 와이파이 이름 (2.4GHz 만 가능!)
const char* WIFI_PASS = "비밀번호"; // ★ 와이파이 비밀번호
const char* URL = "https://script.google.com/macros/s/XXXX/exec"; // ★ Apps Script 배포 주소 (/exec로 끝나는지
확인!)

// 센서·모듈 객체 생성
DHT dht(DHTPIN, DHT11); // 온습도 센서 (핀, 종류)
Adafruit_NeoPixel led(NUMLED, LEDPIN, NEO_GRB + NEO_KHZ800); // 네오픽셀
LiquidCrystal_I2C lcd(0x27, 16, 2); // LCD (주소 0x27, 16칸×2 줄 — 안 나오면 0x3F)

// 팬 두 개를 켜고(true) 끄는(false) 함수
void fan(bool on) {
    // "on ? HIGH : LOW" = 조건 연산자: on이 참이면 HIGH, 거짓이면 LOW
    digitalWrite(AA, on ? HIGH : LOW); digitalWrite(AB, LOW); // 팬 A
    digitalWrite(BA, on ? HIGH : LOW); digitalWrite(BB, LOW); // 팬 B
}

// 네오픽셀을 생장등(보라색)으로 켜고(true) 끄는(false) 함수
void warmLight(bool on) {
    for (int i = 0; i < NUMLED; i++) // LED 12 개를 하나씩
        led.setPixelColor(i, on ? led.Color(255, 0, 255) : 0); // 켜면 보라색(생장등 색!), 끄면 0
    led.show(); // 실제 LED에 반영
}

// 주소(url)로 GET 요청을 보내고, 서버의 응답 문자열을 받아오는 함수
String httpGET(String url) {
    HTTPClient http;
    http.begin(url); // 통신 준비
    http.setFollowRedirects(HTTPC_FORCE_FOLLOW_REDIRECTS); // 구글의 주소 자동이동(리다이렉트)을 따라가기 — 필수!
    http.GET(); // 요청 발사!
    String body = http.getString(); // 서버가 보낸 응답 본문 읽기
    http.end(); // 통신 마무리
    body.trim(); // 응답 앞뒤의 공백·줄바꿈 제거
    return body; // 결과 돌려주기
}

// 처음 한 번만 실행되는 초기화 구간
void setup() {
    Serial.begin(115200); // 시리얼 통신 시작
    pinMode(AA, OUTPUT); pinMode(AB, OUTPUT); // 모터드라이버 핀 4 개를 출력용으로
    pinMode(BA, OUTPUT); pinMode(BB, OUTPUT);
    dht.begin(); // 온습도 센서 시작
    led.begin(); // 네오픽셀 시작
    led.setBrightness(150); // 밝기 150 (최대 255)
    lcd.init(); // LCD 초기화
    lcd.backlight(); // 백라이트 켜기
    lcd.print("WiFi..."); // 연결 중 표시

    WiFi.begin(WIFI_SSID, WIFI_PASS); // 와이파이 연결 시도
    while (WiFi.status() != WL_CONNECTED) { // 연결될 때까지 기다리기
        delay(500);
        Serial.print("."); // 0.5초마다 점 찍기
    }
    Serial.println(" WiFi OK!"); // 연결 성공!
    lcd.clear();
    lcd.print("Smart Farm OK!"); // LCD에도 성공 표시
}

// — 최근 상태를 담아두는 변수들 (loop가 돌 때마다 새로 만들지 않고 계속 기억) —
float t = 0, h = 0; // 온도·습도
int soilPct = 0; // 토양습도(%)
String cmd = "AUTO"; // 현재 명령 (시작은 자동 모드)
int dryLimit = 30; // 급수 알림 기준(%) — 앤 슬라이더로 바뀌어요
unsigned long lastRead = 0, lastPoll = 0, lastUpload = 0; // 세 가지 일의 "마지막 실행 시각"

// 서버 응답("AUTO,30" 모양)에서 명령·기준값을 꺼내 적용하는 함수
// — 명령 폴링과 업로드가 같은 모양으로 응답하므로 둘 다 이 함수 하나로!
void handleReply(String r) {
    int comma = r.indexOf(','); // 쉼표(,)의 위치 찾기
    if (comma <= 0) return; // 쉼표가 없으면(통신 실패 등) 이전 값 유지하고 종료
    cmd = r.substring(0, comma); // 쉼표 앞부분 = 명령 (예: "AUTO")
    dryLimit = r.substring(comma + 1).toInt(); // 쉼표 뒷부분 = 기준값 (예: 30, 문자열→정수 변환)

    // 수동 명령이면 바로 실행
    if (cmd == "FAN_ON") fan(true); // 팬 강제 켜기
    if (cmd == "FAN_OFF") fan(false); // 팬 강제 끄기
    if (cmd == "LED_ON") warmLight(true); // 생장등 강제 켜기
    if (cmd == "LED_OFF") warmLight(false); // 생장등 강제 끄기
}

```

```
// 무한 반복 구간 — millis()는 "전원 켜진 뒤 지난 시간(ms)"을 알려주는 시계예요.
// delay와 달리 기다리는 동안 다른 일을 할 수 있어요!
void loop() {
  // — ① 2초마다: 측정 + LCD + 자동 제어 —————
  if (millis() - lastRead >= 2000) {      // 마지막 측정 후 2초가 지났으면
    lastRead = millis();                  // 지금 시각을 기록해 두고
    float nt = dht.readTemperature();     // 새 온도 읽기
    float nh = dht.readHumidity();        // 새 습도 읽기
    if (!isnan(nt)) t = nt;                // 측정 성공했을 때만 저장 (실패=nan이면 이전 값 유지)
    if (!isnan(nh)) h = nh;
    // 2점 보정: 마른 값→0%, 젖은 값→100% (방향이 반대인 센서도 map이 알아서 처리)
    soilPct = constrain(map(analogRead(SOIL), SOIL_DRY_RAW, SOIL_WET_RAW, 0, 100), 0, 100);
    Serial.printf("T %.1f H %.0f Soil %d%% cmd:%s\n", t, h, soilPct, cmd.c_str());

    // 자동 모드일 때만: 더우면·습하면 팬 / 추우면·과습하면 생장등 (05와 같은 규칙!)
    if (cmd == "AUTO") {
      fan((t > TEMP_HIGH) || (h > HUMI_HIGH));      // || 는 "또는"
      warmLight((t < TEMP_LOW) || (soilPct > SOIL_WET));
    }

    // LCD 상황판 — 1줄: 온습도 / 2줄: 토양습도 (+ 급수 알림)
    lcd.clear();                                  // 이전 글자 지우기
    lcd.setCursor(0, 0);                          // 첫째 줄 처음으로
    lcd.print("T:"); lcd.print(t, 1);             // 온도 (소수점 1자리)
    lcd.print(" H:"); lcd.print(h, 0); lcd.print("%"); // 습도 (정수)
    lcd.setCursor(0, 1);                          // 둘째 줄 처음으로
    lcd.print("Soil:"); lcd.print(soilPct); lcd.print("%"); // 토양습도
    if (soilPct < dryLimit) {                      // 기준보다 마르면
      lcd.setCursor(9, 1);                        // 둘째 줄 오른쪽에
      lcd.print("WATER!");                        // 💧 물 주세요! (앱에도 경고가 떠요)
    }
  }

  // — ② 3초마다: 앱 명령 확인 (빠른 반응!) —————
  if (millis() - lastPoll >= 3000) {
    lastPoll = millis();
    handleReply(httpGET(String(URL) + "?mode=cmd")); // 응답: "AUTO,30" (명령,기준값)
  }

  // — ③ 30초마다: 구글 사이트에 기록 —————
  // 업로드 응답에도 "명령,기준값"이 실려 와요 — 통신 1번으로 두 가지 일!
  if (millis() - lastUpload >= 30000) {
    lastUpload = millis();
    lastPoll = millis();                        // 방금 명령도 받았으니 다음 폴링은 3초 뒤로 미루기
    // 주소 뒤에 ?이름=값&이름=값 형태로 측정값을 붙여서 전송
    String up = String(URL)
      + "?soil=" + String(soilPct)
      + "&temp=" + String(t)
      + "&humi=" + String(h);
    String r = httpGET(up);                     // 전송하고 응답 받기
    Serial.println("upload: " + r);              // 응답 확인 (예: "AUTO,30")
    handleReply(r);                             // 응답 속 명령·기준값도 적용
  }
}
```

④ AI Studio 앱 만들기

1. aistudio.google.com → Build → New App
2. 01_docs/ai_studio_prompt.md 의 프롬프트를 복사 → '여기에_내_웹앱_URL' 두 곳을 내 /exec 주소로 바꿔 붙여넣기
3. 앱이 만들어지면: 센서 카드 3개 확인 → 팬/생장등 버튼 → 슬라이더 저장 테스트
4. 폰 브라우저에서 열어 '홈 화면에 추가' → 앱처럼 설치 완료!

📷 사진 자리: 완성된 앱 화면 스크린샷 + 사이트에 기록이 쌓인 화면

💡 앱 버튼 → 실제 동작까지 3~6 초는 정상이에요. 앱이 사이트에 명령을 쓰면, 보드가 3초마다 확인해 실행하는 '폴링' 구조거든요.

⑤ 최종 점검표

점검 항목	확인
시리얼·LCD 에 온도/습도/토양습도가 나온다	☐

구글 시트에 30 초마다 새 줄이 쌓인다	☐
앱에서 센서 값 3 개가 보인다 (10 초 자동 갱신)	☐
앱의 팬/생장등 버튼이 동작한다 (3~6 초 내)	☐
자동모드(AUTO)에서 규칙대로 스스로 동작한다	☐
토양습도가 기준 아래로 가면 앱·LCD 에 물주기 경고	☐

부록 ① 강남콩 관찰일지

씨앗을 심고 매일 기록해요. 토양습도·온도·습도는 구글 시트의 기록에서 그대로 가져오면 됩니다 — 이게 데이터의 힘!

날짜	토양습도(%)	온도(°C)	습도(%)	관찰 (씩·잎·키)	내가 한 일 (급수 등)

강남콩 발아 정보: 발아 기간 3~7 일 · 적정 온도 21.6~25.8°C · 습도 50% 안팎 · 배수 좋은 흙

부록 ② 명령 규칙표 (앱 ↔ 보드 약속)

설정!A1 값	보드 동작
AUTO	자동 모드 — 더우면·습하면 팬 / 추우면·과습이면 생장등
FAN_ON / FAN_OFF	팬 강제 켜기/끄기
LED_ON / LED_OFF	생장등 강제 켜기/끄기

부록 ③ 자주 막히는 문제 해결소

증상	해결
포트가 사용 중/없음	시리얼 모니터 닫기 → USB 재연결 → 새 COM 번호 선택
토양습도가 0%/100% 고정	① 2 점 보정값 확인 ② 신호선이 A2 줄 S 칸인지 ③ 물 담근 직후엔 마를 때까지 값이 높아요
LCD 가 안 나와요	① 보드 SDA(21)·SCL(22) 직결인지 ② 뒷면 파란 다이얼(대비) ③ 주소 0x27↔0x3F
팬이 안 돌아요	배터리(외부 전원) 연결, 모터드라이버 결선(빨강·검정) 확인
WiFi 연결 안 됨	2.4GHz 인지 확인(5GHz 불가), 비밀번호 8 자 이상
시트에 기록이 안 쌓여요	URL 이 /exec 로 끝나는지, 브라우저 테스트 5 종 먼저, 배포 액세스 '모든 사용자'
앱 버튼이 안 먹어요	시트 설정!A1 이 바뀌는지 먼저 확인 → 바뀌면 보드 문제, 안 바뀌면 앱 문제
서버 코드 고쳤는데 그대로예요	배포 관리 → 🖋️ → 새 버전 → 배포 (저장만으론 반영 안 됨!)